
Apex

Desktop Telemetry & AI Driver Coaching
Feature review for supervisor meeting

University of Bristol MSc × IBM — F1 Telemetry & Driver Coaching project

TORCS 1.3.9 • PySide6 / Qt 6 • IBM Granite 4.1 (local)

Dany

18 August 2026

What Apex is

One desktop application covering the whole loop: drive, record, analyse, explain

- A **native desktop app** (PySide6/Qt 6 + PyQtGraph), running on macOS, Linux and Windows from one code base.
- It is the **participant-facing launcher** for the driving study: no terminal, no CSV handling, no simulator configuration.
- **IBM Granite 4.1** runs locally as a race engineer. It reads telemetry evidence and returns advice — it never controls the car.
- Every model claim is **checked against the telemetry** before it is allowed on screen, and the whole exchange is written to an audit record.

Design rule

The application never computes telemetry truth. It renders what `f1coach-core` returns.

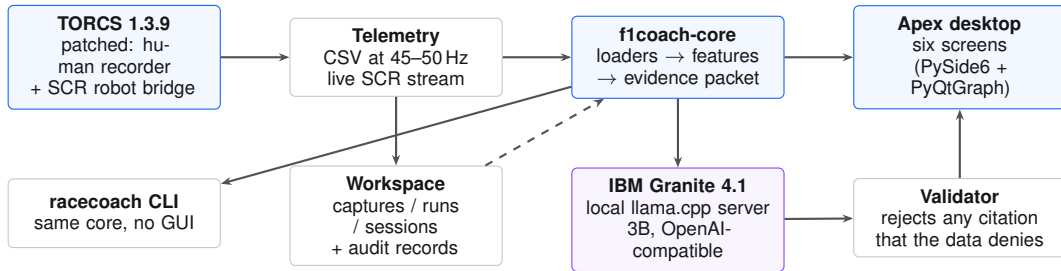
Safety rule

The advisory model is not in the control loop. Deterministic code keeps control of the car at all times.

364 automated tests passing • lint clean

System architecture

One process for the UI and the analysis core; the model sits beside the loop, never inside it



The dashed path is re-analysis: a stored run is reloaded and re-examined without touching the simulator.

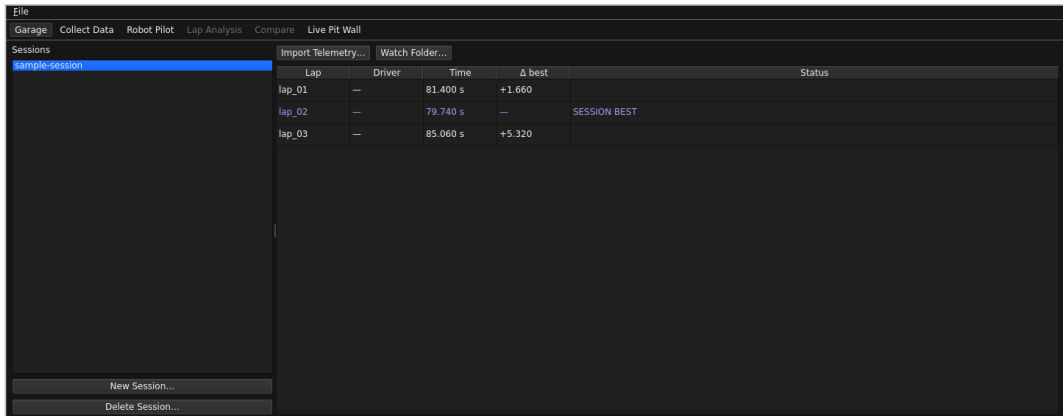
The six screens

Everything below is a live screenshot of the current build

Screen	Who uses it	What it does
Garage	Everyone	Session library, lap table, import and watched folders
Collect Data	Participant	Guided three-step capture; opens the assigned TORCS race
Robot Pilot	Facilitator	Unattended synthetic reference batches (pipeline testing)
Lap Analysis	Everyone	Traces, corner table, driver debrief, Granite race engineer
Compare	Everyone	Two laps on a shared distance axis with cumulative delta
Live Pit Wall	Operator	Live telemetry, tyre state and validated Granite advice

Robot Pilot is hidden unless `APEX_RESEARCH_MODE=1`; participants never see it.

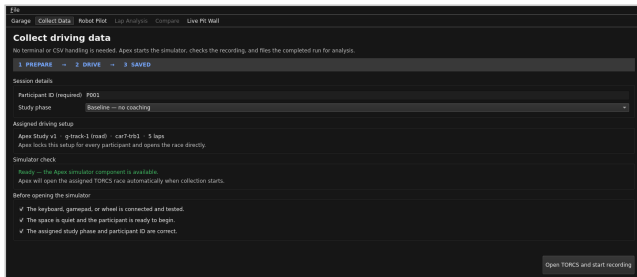
Garage — the session library



First run opens a bundled sample session — no credentials, no network. Purple marks the session best; deltas are measured against it. Telemetry arrives by button, drag-and-drop or a watched folder.

Collect Data — prepare

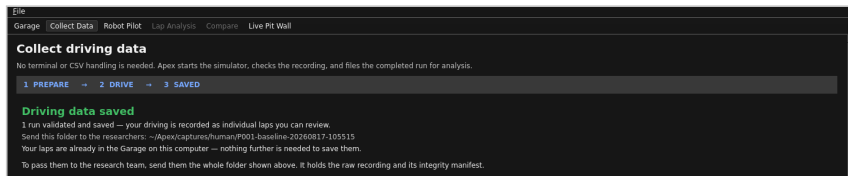
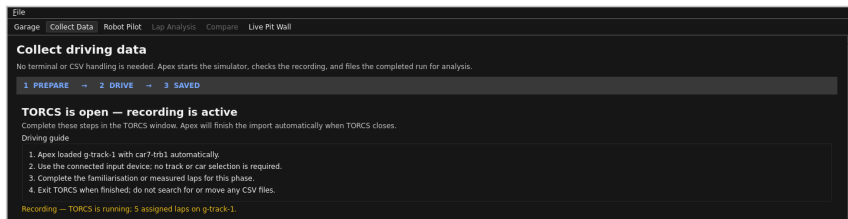
Pseudonymous identity, a frozen study preset, and an explicit simulator check



- **Pseudonymous ID only** — the field explicitly warns against names and email addresses.
- **Frozen preset** apex-study-v1: g-track-1 (road), car7-trb1, five laps, fixed render size.
- **Simulator check** runs before the start button is enabled, so a broken install is caught before a participant sits down.
- **Readiness checklist** covers the input device, the environment and the phase/ID assignment.

Collect Data — drive, then hand over

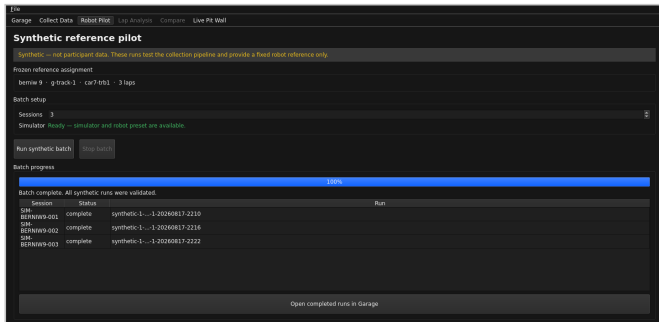
No terminal, no CSV selection, no simulator configuration at any point



Apex opens the assigned five-lap race directly and finishes the import when TORCS closes. The final page ends with the two actions a participant needs — *Collect another session* and *View my laps* — and names the folder to send to the researchers.

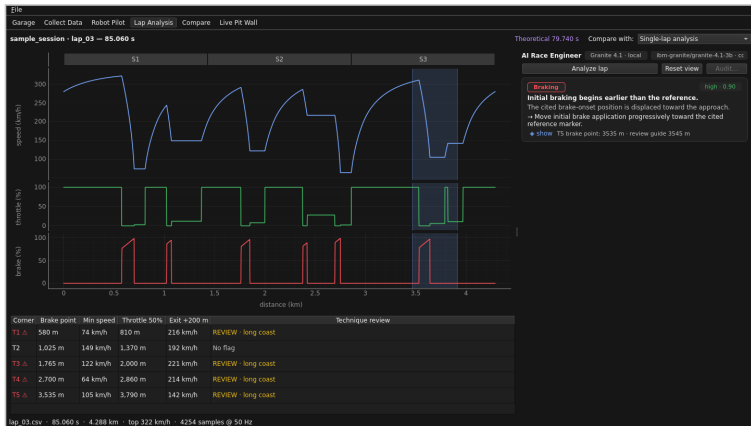
Robot Pilot — validating the pipeline before recruitment

Facilitator-only; synthetic runs are never presented as participant data



- Runs a **pinned built-in robot** (berniw index 9) on the same track and car, three laps per session.
- 1–20 sequential unattended sessions; each is validated and imported exactly like a human capture.
- Stored separately under captures/synthetic/ with synthetic-robot provenance and SIM-BERNIW9-NNN ids.
- The banner states the limit plainly: these runs test the collection pipeline; they cannot show that coaching changes human behaviour.

Lap Analysis — the AI race engineer



Sector ribbon, synced speed/throttle/brake strips on one distance axis, and a corner table with deterministic technique flags. The card is a real response from the local Granite 4.1: focus, action, confidence and a clickable citation — “show” zooms to the cited stretch.

How a model claim earns its place on screen

The interesting engineering is what happens between the model and the display

1. f1coach-core computes an **evidence packet** from the lap: corner spans, brake points, minimum and exit speeds, coasting, pedal overlap.
2. Granite receives **only that packet** and must answer in a fixed JSON schema: issue, evidence, cause, action, confidence.
3. A Python **validator** recomputes every citation. If a corner, value, unit or distance span does not match, the finding is rejected rather than displayed.
4. The accepted report, the prompt and the raw response are written to a **coaching audit record**.
5. Re-opening the same lap and reference **restores** that report — after revalidating the saved evidence again.

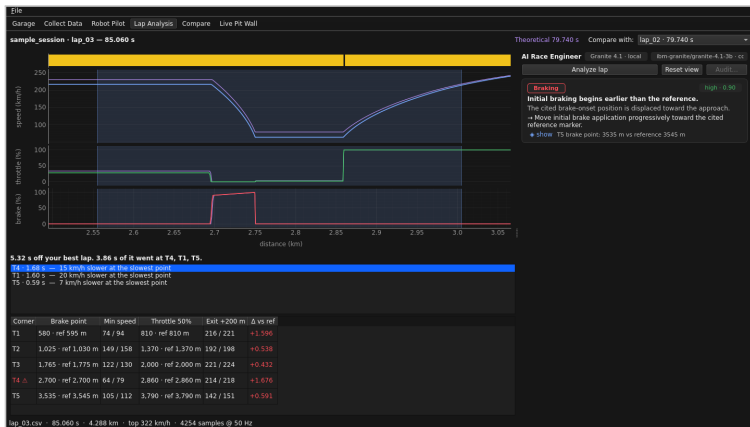
Why it matters

The model is treated as an untrusted narrator of numbers that the application already knows. A hallucinated corner or a wrong unit cannot reach the driver, and every claim that does reach them can be traced back to the exact samples behind it.

Current contract

```
ibm-granite/granite-4.1-3b
post-lap prompt coach-v4
live prompt granite-torcs-live-v1
```

Driver debrief — where the time actually went



Computed deterministically, with or without a model. A reference lap adds the debrief (“5.32 s off your best lap; 3.86 s of it went at T4, T1, T5”), a delta column and the reference trace in purple. Each point pairs a measured time loss with the largest measured difference over the same stretch — and stops short of claiming the second caused the first.

Replay — the same stretch, on the track

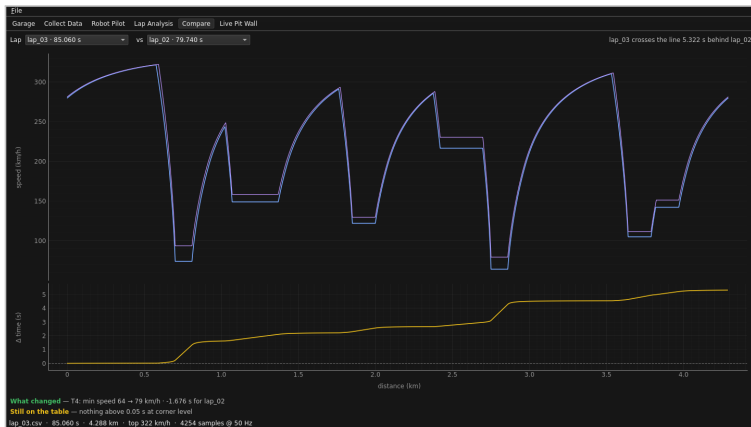
Opens beside the traces so a driver can watch and read at once



- Double-clicking a debrief point replays that stretch in a separate, non-modal window.
- The line is drawn from **recorded world position**; the highlighted segment is the stretch under discussion and the marker is the cursor.
- The readout follows the cursor: elapsed time, speed, throttle, brake and gear.
- A lap recorded without world position degrades to the distance-based view instead of drawing a broken map.

Shown here on a real captured drive, which is the data that carries world position. The bundled demo fixture does not.

Compare — two laps, one distance axis



Also the before/after view for the coaching evaluation. Speed traces overlaid with the cumulative time delta underneath: the slope shows *where* the gap opens, not merely that it exists. The footer separates what changed from what is still on the table, corner by corner.

Live Pit Wall — advice while the car is running

The screenshot shows the 'Live Pit Wall' interface with a dark theme. At the top, there's a menu bar with 'File', 'Garage', 'Collect Data', 'Robot Pilot', 'Lap Analysis', 'Compare', and 'Live Pit Wall'. Below the menu, the title 'Live Pit Wall' is followed by a subtitle 'Granite gives evidence-grounded advice; deterministic code keeps control of the car.' There are two buttons: 'Start live session' and 'Stop safely'. Below this, there's a section for 'Connections' showing 'Bridge Online -- Granite Bridge, SCR 3001' and 'Model Online -- itm-granite/granite-4.1-3b'. The 'Live telemetry' section displays 'Lap 1', 'Simulation time 72.7 s', and 'Speed 156.5 km/h'. A table shows tyre data for FR, FL, RR, and RL wheels, including wear, temperature, pressure, and graining. The 'Latest Granite advice' section has a green 'INFO - throttle' button and text: 'Maintain current pace; tyres are in good condition with low wear and temperature.' It also shows 'Source lap 1 · sim 72.7 s · latency 2.60 s' and 'tire_wear_front_right = 0.002106 ratio (0=new, 1=warn)'. At the bottom, it says 'Recording to --\Apex\runs\live-20260815-215402\telemetry.csv' and 'lap_03.csv · 85.060 s · 4.288 km · top 322 km/h · 4254 samples @ 50 Hz'.

File Garage Collect Data Robot Pilot Lap Analysis Compare Live Pit Wall

Live Pit Wall

Granite gives evidence-grounded advice; deterministic code keeps control of the car.

Advisor: Granite 4.1 (local) Interval: 10.0 s Start live session Stop safely

Connections

Bridge Online -- Granite Bridge, SCR 3001
Model Online -- itm-granite/granite-4.1-3b

Live telemetry

Lap **1** Simulation time **72.7 s** Speed **156.5 km/h**

Tyres · protocol order FR / FL / RR / RL

Wheel	Wear	Temperature	Pressure	Graining
FR	0.2%	32.6 °C	247.2 kPa	0.0%
FL	0.2%	32.6 °C	247.2 kPa	0.0%
RR	0.4%	34.3 °C	234.9 kPa	0.0%
RL	0.4%	34.3 °C	234.9 kPa	0.0%

Latest Granite advice

INFO · throttle

Maintain current pace; tyres are in good condition with low wear and temperature.

Source lap 1 · sim 72.7 s · latency 2.60 s

tire_wear_front_right = 0.002106 ratio (0=new, 1=warn)

Recording to --\Apex\runs\live-20260815-215402\telemetry.csv
lap_03.csv · 85.060 s · 4.288 km · top 322 km/h · 4254 samples @ 50 Hz

Bridge health, tyre state and a real Granite 4.1 response with its measured latency. Advice is asynchronous and latest-only: a slow model delays advice, never the car. Stale advice is labelled rather than silently shown, and the evidence box lists the values it may rest on.

Audit — the record behind every answer

Prompt, raw response, evidence packet and outcome, in one file per run

```
/tmp/claude-567973f-home-hh25303-IBM/b6b21c8b-3f0b-4c1d-9a46-7cbce6dafba/scratchpad/ws/coaching/20260818-112321-granite.json
{
  "written_at": "2026-08-18T11:23:21+00:00",
  "provider": "granite",
  "ok": true,
  "error": null,
  "lap": "lap_03",
  "reference": "lap_02",
  "model": "ibm-granite/granite-4.1-3b",
  "prompt_version": "coach-v4",
  "evidence_summary": {
    "analysis_mode": "comparison",
    "lap": {
      "name": "lap_03",
      "lap_time_s": 85.06
    },
    "reference": {
      "name": "lap_02",
      "lap_time_s": 79.74
    }
  },
  "total_delta_s": 5.32,
  "corners": [
    {
      "corner": "T1",
      "apex_m": 755.0,
      "span_m": [
        505.0,
        955.0
      ],
      "entry_speed_kmh": 319.6,
      "ref_entry_speed_kmh": 319.9,
      "min_speed_kmh": 73.9,
      "ref_min_speed_kmh": 93.6,
      "min_speed_point_m": 705.0,
      "ref_min_speed_point_m": 705.0,
      "exit_speed_kmh": 216.4,
      "ref_exit_speed_kmh": 221.3,
      "brake_point_m": 580.0,
      "ref_brake_point_m": 595.0,
      "peak_brake_pct": 97.9,
      "ref_peak_brake_pct": 97.3,
      "brake_release_m": 705.0
    }
  ]
}
```

- Written for **every** attempt, including failures — the record above names the provider, the model, the prompt version and the outcome.
- Contains the full evidence packet, so a reviewer can recompute what the model was shown.
- Reachable from the analysis screen through the Audit... button; nothing has to be reconstructed from logs.
- This is what makes the coaching claims examinable rather than merely plausible.

Research data pipeline

Raw evidence and analysis copies are kept apart on purpose

Location	Holds	Guarantee
<code>captures/human/</code>	Raw CSV + <code>manifest.json</code>	Participant id, phase, exact TORCS command, study preset, SHA-256 per file
<code>captures/synthetic/</code>	Robot reference batches	Marked <code>synthetic-robot</code> ; never mixed with participant data
<code>runs/</code>	Validated copies + <code>meta.json</code>	Sample count, columns, laps seen, cadence; what analysis reads
<code>sessions/</code>	Canonical per-lap CSV + coaching audits	What the Garage lists and the Analysis screen opens

Telemetry schema `apex-human-v2`: more than 100 raw or directly exposed TORCS channels at roughly 45–50 Hz, including per-wheel wear, temperature, pressure and graining. The whole tree relocates with `APEX_WORKSPACE` so study data can sit on an approved volume.

The same core without a GUI

racecoach — for automation, recovery and reproducible checks

Command	Purpose
<code>import / list</code>	Import a TORCS exporter CSV; list stored runs
<code>analyze</code>	Compute metrics for a stored run
<code>coach</code>	Post-run LLM feedback, optionally grounded in a code overview
<code>run</code>	Drive the car and capture telemetry, with an optional live advisor
<code>capture-human</code>	Facilitator fallback for participant capture
<code>capture-synthetic</code>	Pinned unattended robot reference sessions
<code>report</code>	Render the post-race HTML report

The desktop app, the CLI and the notebooks are three faces of one package: nothing in the UI can disagree with what the CLI computes. A contract-only demo needs no model download at all: `python -m racecoach.granite.smoke -mock`.

Engineering status

Where the build stands today

Done

- All six screens implemented and under test.
- Native TORCS 1.3.9 overlay: human telemetry recorder, SCR robot bridge, locked study preset.
- Local Granite 4.1 integration end to end, post-lap and live, with validation and audit records.
- Human capture, synthetic reference capture, and the full storage contract.
- Windows build and per-user installer; macOS and Linux from the same source.
- **364 automated tests passing**, lint clean.

Open

- Pilot acceptance run with a real participant, end to end.
- Captures so far are short pilot drives; no multi-lap human dataset yet, so no before/after coaching result can be claimed.
- World position is recorded by the human capture path but not by the live SCR path, so the replay map is limited to captured drives.
- Race Engineer Q&A panel remains a stretch goal.
- Report export exists; packaging it into the study protocol is next.

Next steps

What I propose to do before the next review

1. **Run the pilot acceptance session** end to end on the Windows build, with a real participant and the frozen `apex-study-v1` preset — this is the last unproven link in the chain.
2. **Collect a multi-lap baseline dataset** so that Compare can finally be used for what it was built for: before and after coaching.
3. **Add world position to the live SCR path** so replay works for live sessions as well as captured ones.
4. **Freeze a versioned study preset** after pilot evidence and supervisor approval, rather than continuing to develop against `apex-study-v1`.

Question for this meeting: is the pilot protocol ready to be run as it stands, or should the preset change first?

Note on the screenshots

So the evidence in this deck is itself auditable

- Every screenshot is the current build, rendered headless from the repository at commit d33350d on 18 August 2026.
- The coaching content on the Analysis and Live Pit Wall slides is a **real response** from the local `ibm-granite/granite-4.1-3b` server, not a mock or a canned reply. Latency shown is measured.
- The Garage, Analysis, Debrief and Compare screens use the bundled three-lap sample session, which ships with the application.
- The Replay screen uses a real captured drive, because that is the data that carries world position.
- On the Collect Data and Robot Pilot slides the **screens are real** but are shown in their completed states; running them for the screenshot would have required a live TORCS session and a full synthetic batch.